



KAZAKH-BRITISH
TECHNICAL
UNIVERSITY

Programming Principles I

Lecture 3



Zhiger Bolatov

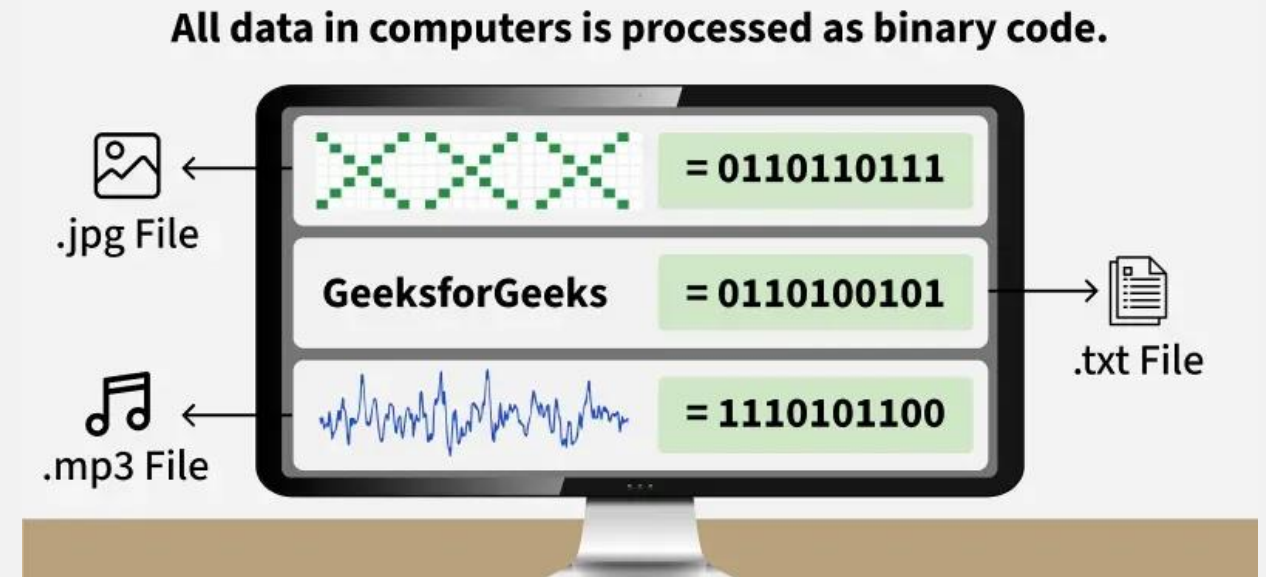
Senior Lecturer
School of IT&I

- Numeric Systems
(Binary, Octal, Decimal, Hexadecimal)
- Bitwise operators
(AND, OR, XOR, NOT, left shift, right shift)
- Bitwise masking

Numeric Systems

Number system is a method of representing numbers with the help of a set of symbols and rules; it is a mathematical notation used to represent quantities or values in various forms.

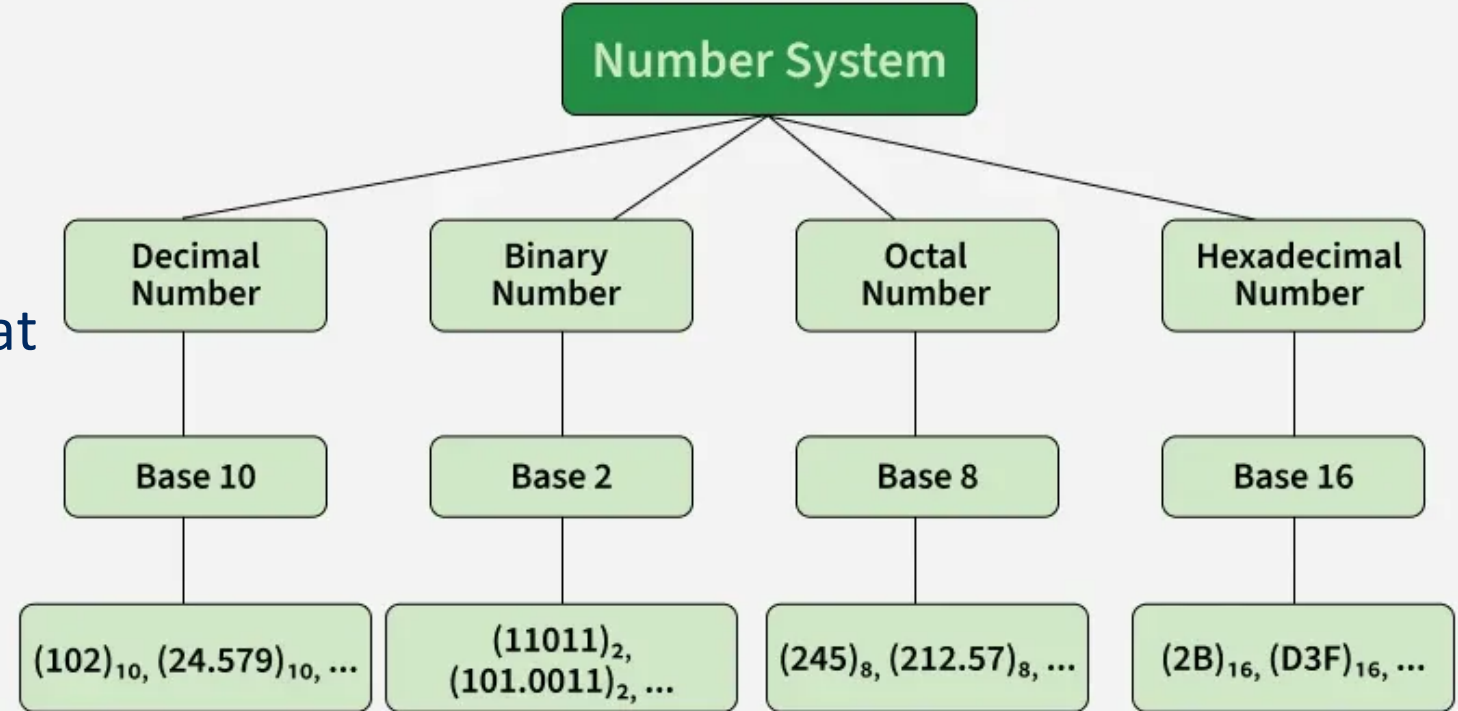
- Humans understand decimal, and computers understand binary, so we have two different number systems.
- Binary can be long for a large amount of data, so there are more systems like octal and hexadecimal that allow binary sequences to be broken into groups, making them less error-prone and more readable compared to binary.



Numeric Systems

Number systems are of various types based on their base value and the number of allowed digits.

- Decimal (base 10): the digits 0–9 — what humans use.
- Binary (base 2): only 0 and 1 — what the hardware actually stores.
- Octal (base-8): the digits 0–7 — commonly used to simplify the representation of binary data.
- Hexadecimal (base 16): digits 0–9 then A–F — a compact way to write binary (each hex digit = 4 bits).



C++ literals: 0b1010 (binary), 0x1A (hex), plain 26 (decimal).

Numeric Systems

Number System		Base	Used Digits
1	Binary System	2	0, 1
2	Octal System	8	0,1,2,3,4,5,6,7
3	Decimal System	10	0,1,2,3,4,5,6,7,8,9
4	Hexadecimal System	16	0,1,2,3,4,5,6,7,8,9
			A,B,C,D,E,F

Binary -> Decimal

A binary number is converted into a decimal number by multiplying each digit of the binary numbers 1 or 0 to the corresponding to the power of 2 according to the place value

Binary to Decimal Conversion

$$\begin{array}{cccccccc} 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 \\ (11101011)_2 & \longrightarrow & (?)_{10} \end{array}$$

$$1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$128 + 64 + 32 + 0 + 8 + 0 + 2 + 1$$

$$(235)_{10}$$

Octal -> Decimal

Octal numbers are represented in base 8, but the decimal numbers are of base 10
Hence, to convert an octal number to a decimal number, the base of that number
is to be changed

Octal to Decimal Conversion

$$(247)_8 \longrightarrow (?)_{10}$$

$$2 \times 8^2 + 4 \times 8^1 + 7 \times 8^0$$

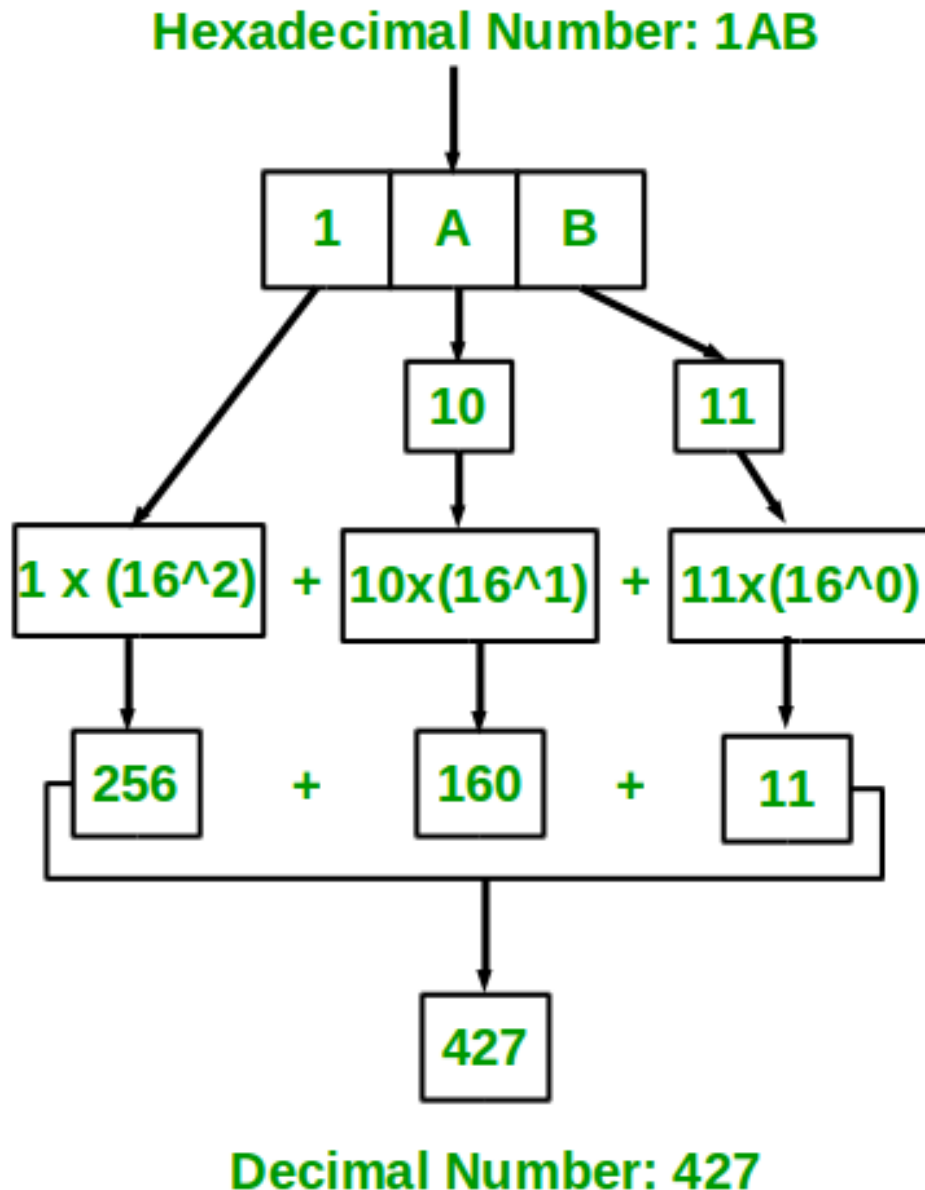
$$2 \times 64 + 4 \times 8 + 7$$

$$128 + 32 + 7$$

$$(167)_{10}$$

Hexadecimal -> Decimal

To convert a hexadecimal number to decimal (base-10), multiply each digit by its corresponding power of 16 and sum the results



Decimal -> Binary

$(243)_{10} \longrightarrow (?)_2$

2	243	1
2	121	1
2	60	0
2	30	0
2	15	1
2	7	1
2	3	1
	1	

$(11110011)_2$

Binary numbers use base-2. To convert a decimal number to binary, follow the steps given below:

- Step 1: Divide the Decimal Number with the base of the number system to be converted to. Here the conversion is to binary, hence the divisor will be 2.
- Step 2: The remainder obtained from the division will become the least significant digit of the new number.
- Step 3: The quotient obtained from the division will become the next dividend and will be divided by the base i.e. 2.
- Step 4: The remainder obtained will become the second least significant digit i.e. it will be added to the left of the previously obtained digit.

Decimal -> Octal

$$(243)_{10} \longrightarrow (?)_8$$

8	243	3
8	30	6
	3	

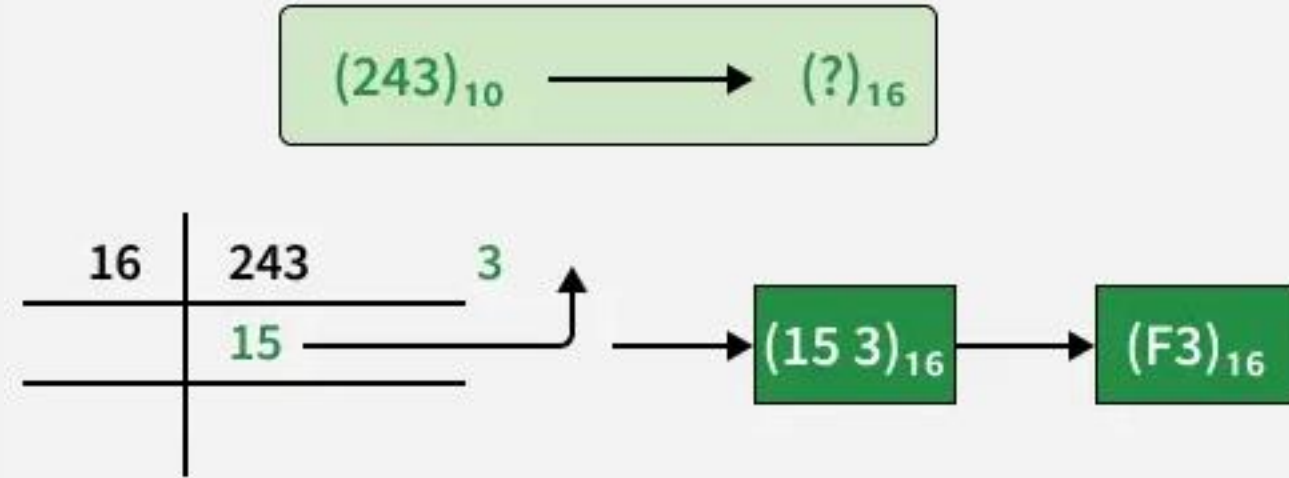
→ (363)₈

Octal numbers use base-8. Here's the process:

- Step 1: Divide the Decimal Number with the base of the number system to be converted to. Here the conversion is to octal, hence the divisor will be 8.
- Step 2: The remainder obtained from the division will become the least significant digit of the new number.
- Step 3: The quotient obtained from the division will become the next dividend and will be divided by base i.e. 8.
- Step 4: The remainder obtained will become the second least significant digit i.e. it will be added to the left of the previously obtained digit.

Decimal -> Hexadecimal

Hexadecimal uses base-16, with digits 0–9 and A–F (where A = 10, B = 11, C = 12, D = 13, E = 14, F = 15). Follow these steps:



Step 1: Divide the Decimal Number with the base of the number system to be converted to. Here the conversion is to Hex hence the divisor will be 16.

Step 2: The remainder obtained from the division will become the least significant digit of the new number.

Step 3: The quotient obtained from the division will become the next dividend and will be divided by base i.e. 16.

Step 4: The remainder obtained will become the second least significant digit i.e. it will be added to the left of the previously obtained digit.

Bitwise operators are special operators in programming that work directly on binary bits (0 and 1). Since computers store all data in binary form, bitwise operations help us manipulate data at the lowest level using operations like AND, OR, XOR, NOT, and bit shifting.

- They are commonly used in optimization, performance-critical code, masking, toggling bits, and low-level programming.
- They are used to perform fast calculations and binary manipulation.
- They help in setting, clearing, checking, and toggling bits.
- Bitwise operations are often faster for low-level and bit-manipulation tasks.
- Bitwise operations enable packing multiple flags into a single variable, reducing memory usage.

Bitwise operators

Bit manipulation works on binary bits (0 and 1) using bitwise operators, making it fast and efficient. These operations are executed by the CPU's Arithmetic Logic Unit (ALU) and are commonly used for optimization, efficient flag handling, and performance-critical tasks.

Bitwise Operator Truth Table

The main bitwise operators are:

- AND (&)
- OR (|)
- XOR (^)
- NOT (~)
- Left Shift (<<)
- Right Shift (>>)

X	Y	X&Y	X Y	X^Y	~(X)
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Bitwise AND Operator (&)

The bitwise AND operator is denoted using a single ampersand symbol, i.e. &. The & operator takes two equal-length bit patterns as parameters. The two-bit integers are compared. If the bits in the compared positions of the bit patterns are 1, then the resulting bit is 1. If not, it is 0.

Truth Table of Bitwise AND (&) Operator

X	Y	X&Y
0	0	0
0	1	0
1	0	0
1	1	1

Bitwise AND Operator (&)

Example:

Take two bit values X and Y, where $X = 7 = (111)_2$ and $Y = 4 = (100)_2$.

Take Bitwise AND of both X & Y

$$\begin{array}{r} 111_2 \\ \& 100_2 \\ \hline 100_2 = 4 \end{array}$$

```
int a = 7, b = 4;  
int result = a & b;  
cout << result << endl;
```

Bitwise OR Operator (|)

The | Operator takes two equivalent length bit designs as boundaries; if the two bits in the looked-at position are 0, the next bit is zero. If not, it is 1.

Truth Table of Bitwise OR (|) Operator

X	Y	X Y
0	0	0
0	1	1
1	0	1
1	1	1

Bitwise OR Operator (|)

Example:

Take two bit values X and Y, where $X = 7 = (111)_2$ and $Y = 4 = (100)_2$.

Take Bitwise OR of both $X | Y$

$$\begin{array}{r} 111_2 \\ | 100_2 \\ \hline 111_2 = 7 \end{array}$$

```
int a = 7, b = 4;  
int result = a | b;  
cout << result << endl;
```

Bitwise XOR Operator (^)

The ^ operator (also known as the XOR operator) stands for Exclusive Or. Here, if bits in the compared position do not match their resulting bit is 1. i.e, The result of the bitwise XOR operator is 1 if the corresponding bits of two operands are opposite, otherwise 0.

Truth Table of Bitwise XOR (^) Operator

X	Y	$X \wedge Y$
0	0	0
0	1	1
1	0	1
1	1	0

Bitwise XOR Operator (^)

Example:

Take two bit values X and Y, where $X = 7 = (111)_2$ and $Y = 4 = (100)_2$.

Take Bitwise XOR of both $X \wedge Y$

$$\begin{array}{r} 1\ 1\ 1_2 \\ \wedge\ 1\ 0\ 0_2 \\ \hline 0\ 1\ 1_2 = 3 \end{array}$$

```
int a = 7, b = 4;  
int result = (a ^ b);  
cout << result << endl;
```

Bitwise NOT Operator (~)

All the above three bitwise operators are binary operators (i.e, requiring two operands in order to operate). Unlike other bitwise operators, this one requires only one operand to operate.

- The bitwise Not Operator takes a single value and returns its one's complement.
- The one's complement of a binary number is obtained by toggling all bits in it, i.e, transforming the 0 bit to 1 and the 1 bit to 0.

Truth Table of Bitwise NOT (~) Operator

X	$\sim(X)$
0	1
1	0

Bitwise NOT Operator (~)

Example:

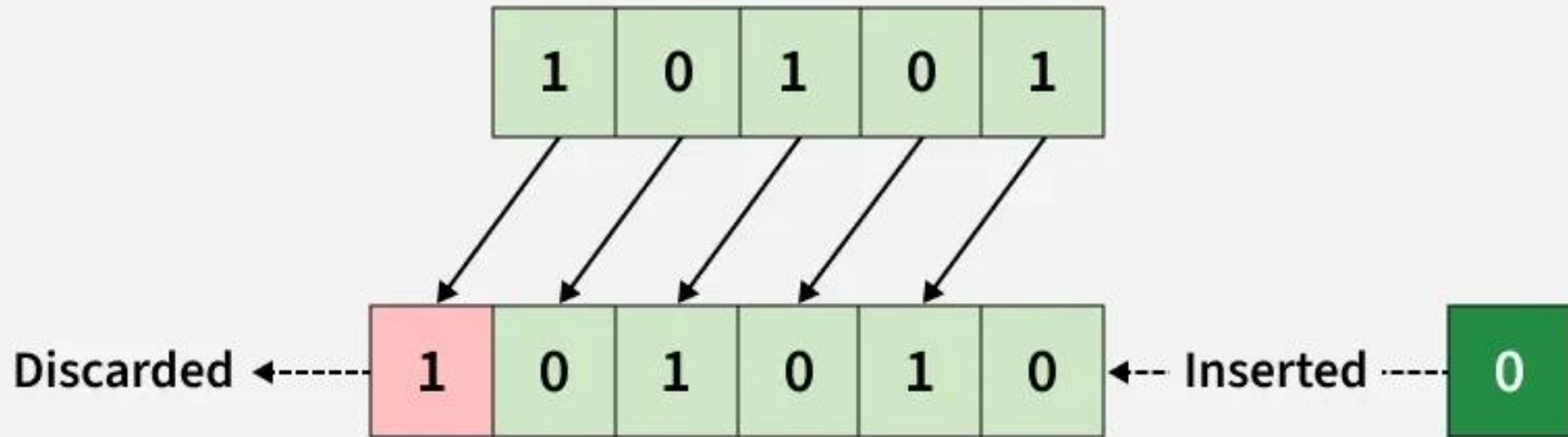
Take a 4-bit value X , where $X = 9 = (1001)_2$. Take the Bitwise NOT of X .

$$\begin{array}{r} \sim 1\ 0\ 0\ 1_2 \\ \hline 0\ 1\ 1\ 0_2 = 6 \end{array}$$

```
int a = 9;  
int result = (~a);  
cout << result << endl;
```

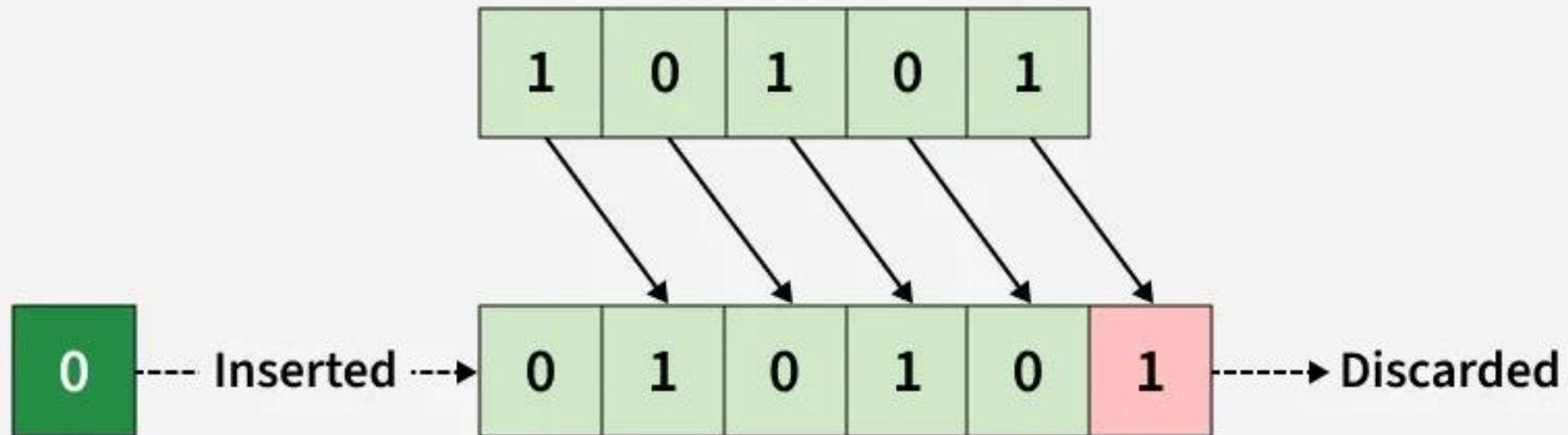
Bitwise Left-Shift (<<)

The left shift operator is denoted by the double left arrow key (<<). The general syntax for left shift is shift-expression << k. The left-shift operator causes the bits in shift expression to be shifted to the left by the number of positions specified by k. The bit positions that the shift operation has vacated are zero-filled.



Bitwise Right-Shift (>>)

The right shift operator is denoted by the double right arrow key (>>). The general syntax for the right shift is "shift-expression >> k". The right-shift operator causes the bits in shift expression to be shifted to the right by the number of positions specified by k. For unsigned numbers, the bit positions that the shift operation has vacated are zero-filled. For signed numbers, the sign bit is used to fill the vacated bit positions. In other words, if the number is positive, 0 is used, and if the number is negative, 1 is used.



Application of Bit Operators

- Bit operations are used for the optimization of embedded systems.
- The Exclusive-OR (XOR) operator can be used to confirm the integrity of a file, making sure it has not been corrupted, especially after it has been in transit.
- Bitwise operations are used in Data encryption and compression.
- Bits are used in the area of networking, framing the packets of numerous bits which are sent to another system generally through any type of serial interface.
- Digital Image Processors use bitwise operations to enhance image pixels and to extract different sections of a microscopic image.

Power of 2

Powers of two		
k	2^k	Binary representation
0	$2^0 = 1$	000 1
1	$2^1 = 2$	00 1 0
2	$2^2 = 4$	0 1 00
3	$2^3 = 8$	1 000
4	$2^4 = 16$	1 0000

Examples: $n \& (n - 1) = 0$
(for powers of two)

$n = 16$ (2^4)

$n = 10000$ (16)
 $n - 1 = 01111$ (15)

 $n \& (n - 1) = 00000$ (0)

Result = 0
→ power of two

$n = 8$ (2^3)

$n = 01000$ (8)
 $n - 1 = 00111$ (7)

 $n \& (n - 1) = 00000$ (0)

Result = 0
→ power of two

$n = 4$ (2^2)

$n = 00100$ (4)
 $n - 1 = 00011$ (3)

 $n \& (n - 1) = 00000$ (0)

Result = 0
→ power of two

Set a bit in the number

If we want to set a bit at n-th position in the number 'num', it can be done using the 'OR' operator(|).

- First, we left shift 1 to n position via $(1 \ll n)$.
- Then, use the "OR" operator to set the bit at that position. "OR" operator is used because it will set the bit even if the bit is unset previously in the binary representation of the number 'num'.

```
int num = 4, pos = 1;
```

```
int result = num | (1 << pos);
```

```
cout << result << endl;
```

Set a bit in a number

To set the bit at position n in a number num , we use the OR operator ($|$).

- 1 Left shift 1 by n positions to create a mask: $(1 \ll n)$.
- 2 Use the OR operator to set the bit: $num | (1 \ll n)$.

Why OR?

- $0 | 1 = 1$
- $1 | 1 = 1$

So the selected bit will be set to 1 even if it was previously 0 or 1.

Example: $num = 4$, $pos = 1$

1 Write the number in binary

$num = 4$

7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0

(4 in decimal)

2 Create the mask

$pos = 1$

$1 \ll pos = 1 \ll 1 = 2$

7	6	5	4	3	2	1	0
0	0	0	0	0	0	1	0

(2 in decimal)

3 Use OR operator

$result = num | (1 \ll pos)$

	7	6	5	4	3	2	1	0
num :	0	0	0	0	0	1	0	0
mask :	0	0	0	0	0	0	1	0

result:	0	0	0	0	0	1	1	0
---------	---	---	---	---	---	---	---	---

(6 in decimal)

C++ code:

```
int num = 4, pos = 1;
int result = num | (1 << pos);
cout << result << endl;
```



Output:

6

Clear a bit in the number

Suppose we want to unset a bit at n-th position in number 'num' then we have to do this with the help of "AND" (&) operator.

- First, we left shift '1' to n position via $(1 \ll n)$ then we use bitwise NOT operator '~' to unset this shifted '1'.
- Now after clearing this left shifted '1' i.e making it to '0' we will 'AND'(&) with the number 'num' that will unset bit at n-th position.

```
int num = 7, pos = 1;  
int result = num & ~(1 << pos);  
cout << result << endl;
```

Clear a bit in the number

To **clear (unset)** the bit at position n in a number num , we use the **AND operator** ($\&$).

- 1 Left shift 1 by n positions: $(1 \ll n)$.
- 2 Apply bitwise NOT ($\sim$) to get a mask with 0 at position n .
- 3 Use AND with the number: $num \& (\sim(1 \ll n))$.

Why AND and NOT?

$\sim(1 \ll n)$ creates a mask with 0 at position n and 1 elsewhere.

- $0 \& 0 = 0 \rightarrow$ bit is cleared
- $1 \& 0 = 0 \rightarrow$ bit is cleared
- $0 \& 1 = 0 \rightarrow$ other bits stay the same
- $1 \& 1 = 1 \rightarrow$ other bits stay the same

Example: $num = 7$, $pos = 1$

- 1 Write the number in binary

$num = 7$

7	6	5	4	3	2	1	0
0	0	0	0	0	1	1	1

(7 in decimal)

- 2 Create the mask

$pos = 1$
 $1 \ll pos = 1 \ll 1 = 2$

7	6	5	4	3	2	1	0
0	0	0	0	0	0	1	0

$\sim(1 \ll pos) = \sim 2$

7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	1

(mask in binary)

- 3 Apply AND

	7	6	5	4	3	2	1	0
num :	0	0	0	0	0	1	1	1
mask :	1	1	1	1	1	1	0	1

result:	0	0	0	0	0	1	0	1
---------	---	---	---	---	---	---	---	---

- 4 Result

result = 5

7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1

(5 in decimal)

C++ code:

```
int num = 7, pos = 1;
int result = num & (~ (1 << pos));
cout << result << endl;
```



Output:

5

Checking if the bit is Set or Unset

We used the left shift (<<) operation on 1 to shift the bits to n-th position and then use the & operation with number given number, and check if it is not-equals to 0.

```
int num = 5, pos = 2;  
bool result = num & (1 << pos);  
cout << result << endl;
```

Checking if the bit is Set or Unset

To check if the bit at position n is set (1) or unset (0), we can:

- 1 Left shift 1 by n positions: $(1 \ll n)$ to create a mask.
- 2 Use the AND operator (&) with the number, and check if the result is not equal to 0. If the result is not 0, the bit is set (1); otherwise, it is unset (0).

Key idea

- $1 \ll n$ creates a mask with a 1 at position n .
- $\text{num} \& (1 \ll n)$ extracts the bit at position n .
- If the result is not 0 $\rightarrow$ the bit is set (1).
- If the result is 0 $\rightarrow$ the bit is unset (0).

Example: $\text{num} = 5$, $\text{pos} = 2$

1 Write the number in binary

$\text{num} = 5$

7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1

(5 in decimal)

2 Create the mask

$\text{pos} = 2$

$1 \ll \text{pos} = 1 \ll 2 = 4$

7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0

(4 in decimal)

3 Apply AND and check the result

	7	6	5	4	3	2	1	0
num :	0	0	0	0	0	1	0	1
mask :	0	0	0	0	0	1	0	0

result :	0	0	0	0	0	1	0	0
----------	---	---	---	---	---	---	---	---

result = 4 (not equal to 0) $\rightarrow$ bit is set (1)

C++ code:

```
int num = 5, pos = 2;
bool result = num & (1 << pos);
cout << result << endl;
```

Output:

1
(the bit at position 2 is set)

Count Set bits in an integer

Counting set bits means, counting total number of 1's in the binary representation of an integer. For this problem we go through all the bits of given number and check whether it is set or not by performing AND operation (with 1).

```
int n = 5, count = 0;
while (n > 0)
{
    count += (n & 1);
    n >>= 1;
}
cout << count << endl;
```

Count Set bits in an integer

Counting set bits means counting the number of 1's in the binary representation of an integer. We go through all the bits of the number and check whether each bit is set by performing $(n \& 1)$. After checking, we right shift the number by 1 ($n \gg= 1$) and repeat until n becomes 0.

Key idea

- $(n \& 1)$ checks the least significant bit (rightmost bit).
- If it is 1, we increment the counter.
- $n \gg= 1$ right shifts the number by 1 to check the next bit.
- Repeat until n becomes 0.

Example: $n = 5$

Step	n (decimal)	n (binary)	n & 1	count	Explanation
1	5	0 0 0 0 0 1 0 1	1	1	Last bit is 1 → count = 1
2	2	0 0 0 0 0 0 1 0	0	1	Last bit is 0 → count = 1 (no change)
3	1	0 0 0 0 0 0 0 1	1	2	Last bit is 1 → count = 2
4	0	0 0 0 0 0 0 0 0	–	2	n becomes 0 → stop

C++ code:

```
int n = 5, count = 0;
while (n > 0)
{
    count += (n & 1); // add 1 if last bit is 1
    n >>= 1;         // right shift n by 1
}
cout << count << endl;
```



Output:

2

(5 in binary is 101, which has 2 ones)

Binary representation of 5:

0 0 0 0 0 1 0 1

Number of 1's = 2



KAZAKH-BRITISH
TECHNICAL
UNIVERSITY

Questions?

Lecture 3

z.bolatov@kbtu.kz



Zhiger Bolatov

Senior Lecturer
School of IT&I